

NATIONAL RESEARCH UNIVERSITY
HIGHER SCHOOL OF ECONOMICS

Faculty of Computer Science
Bachelor's Programme "Data Science and Business Analytics"

Software Project Report on the Topic:
A VQA-Augmented Multimodal Meme Retrieval System
(interim, the first stage)

Submitted by the Student:

group #БПАД233, 3rd year of study
Khamidullin Amir Aydarovich

Approved by the Project Supervisor:

Burlova Albina Sergeevna
Senior lecturer
Faculty of Computer Science, HSE University

Contents

Annotation	4
1 Introduction	5
1.1 Motivation and Relevance	5
1.2 Project Goal and Objectives	5
1.3 Practical Significance	6
1.4 Novelty	6
2 Literature Review	7
3 Work Plan	9
4 Methodology	10
4.1 Data Collection	10
4.1.1 OCR Pipeline	10
4.1.2 Data Cleaning and Filtering	11
4.1.3 Final Dataset	12
4.2 VQA Annotation Pipeline	13
4.2.1 Model Selection	13
4.2.2 Subset Selection for Annotation	13
4.2.3 Annotation Schema	14
4.2.4 Pipeline Implementation	15
4.3 System Architecture	16
4.3.1 Embedding Generation	16
4.3.2 Index Construction	16
4.3.3 Search Modes	16
4.3.4 Application Layer	17
5 Current Results	18
5.1 Dataset Summary	18
5.2 VQA Annotation Analysis	18
5.3 Search System: Embedding Indices and Retrieval Demo	20
5.3.1 Text Search Examples	21
5.3.2 Image Search Examples	22

5.3.3 Hybrid Search Example	23
5.4 Completed Milestones	23
Conclusion	24
References	25

Annotation

This work develops an intelligent multimodal meme retrieval system that enables users to find relevant images by text description, reference image, or their combination. The system is based on modern machine learning methods: multimodal embeddings (CLIP), vision-language models for description generation, and a two-stage retrieval architecture. At the current interim stage, data collection, OCR, VQA annotation, embedding generation, FAISS indexing, and a working search demo have been completed. Cross-encoder re-ranking, FastAPI backend, and Telegram bot are planned for the second stage. System quality will be evaluated using information retrieval metrics (Hit@K, Recall@K, MRR@10) on a validation set of 300 manually annotated queries.

Аннотация

В данной работе разрабатывается интеллектуальная система мультимодального поиска мемов, позволяющая пользователям находить релевантные изображения по текстовому описанию, референсной картинке или их комбинации. Система основана на современных методах машинного обучения: мультимодальных эмбедингах (CLIP), визуально-языковых моделях для генерации описаний и двухступенчатой архитектуре поиска с переранжированием. Планируется создание базы из 10 000 мемов с описаниями, построение векторного индекса для быстрого поиска и разработка Telegram-бота как пользовательского интерфейса. Качество системы будет оценено с использованием метрик информационного поиска (Hit@K, Recall@K, MRR@10) на валидационной выборке из 300 запросов с ручной разметкой.

Keywords

Multimodal retrieval, CLIP, vision-language models, VQA, embeddings, vector search, information retrieval, memes, Telegram bot, re-ranking

1 Introduction

1.1 Motivation and Relevance

In the modern digital landscape, internet memes have become a universal means of communication, combining visual content and text to convey cultural context, humor, and social commentary. According to various studies, millions of new memes are created daily, and their use spans all areas of online communication—from personal messaging to marketing campaigns.

However, existing image search tools are insufficiently effective for working with memes. Standard search engines are oriented toward keyword-based or visual similarity search, which does not account for the specifics of memes as multimodal content. The main challenges include the following:

1. Semantic complexity: a meme often contains hidden meaning that cannot be derived directly from the visual content or text separately. Understanding a meme requires joint analysis of the image and overlaid text.
2. Cultural context: meme interpretation presupposes knowledge of cultural references, internet trends, and the specifics of online communities. The same template can be used to convey completely different messages.
3. Format diversity: memes range from simple images with text to complex collages and comics, which complicates the application of unified processing methods.

The emergence of multimodal machine learning models, such as CLIP [13], opens new opportunities for solving the meme retrieval task. These models can create a unified vector representation for text and images, enabling cross-modal search.

1.2 Project Goal and Objectives

The goal of this project is to develop an intelligent multimodal meme retrieval service, implemented as a Telegram bot with a backend system, that allows users to find relevant memes by text description, reference image, or their combination.

To achieve this goal, the following objectives must be accomplished:

1. Collect and prepare a database of approximately 10,000 memes from open sources, including text extraction from images (OCR) and semantic description generation using vision-language models (VQA annotations).

2. Build a multimodal index based on vector embeddings (CLIP) with efficient storage and search organization in a vector database.
3. Implement a two-stage retrieval architecture: fast approximate search by embeddings followed by re-ranking of top-K candidates to improve precision.
4. Develop a Telegram bot supporting three input modes with a user-friendly interface.
5. Conduct system quality evaluation using information retrieval metrics on a validation set with manual annotation.

1.3 Practical Significance

The developed system has practical value for a wide range of users. Regular users will get a convenient tool for quickly finding the right memes in everyday communication. SMM specialists and content managers will be able to promptly find relevant visual content for publications. Researchers of internet culture will receive a tool for analyzing and systematizing memes.

Additionally, the architectural solutions and methods developed within this project can be applied to other multimodal image retrieval tasks.

1.4 Novelty

The novelty of this project lies in its comprehensive approach to multimodal meme retrieval, which includes:

1. Use of VQA annotations: applying vision-language models to generate structured meme descriptions that account for both visual content and overlaid text. Unlike simple captioning, the VQA approach allows extracting deeper semantic information.
2. Two-stage retrieval architecture: combining fast approximate search by embeddings (first stage) with more precise re-ranking through a cross-encoder model (second stage). This achieves a balance between search speed and quality.
3. Hybrid representation: combining dense embeddings (CLIP) and sparse representations (BM25 on texts) to improve search robustness for different query types.
4. Specialization for the meme domain: adapting approaches to the specifics of internet memes, which differ from standard images by having overlaid text, cultural references, and semantic complexity.

2 Literature Review

From existing works in multimodal image retrieval, several key directions can be identified: building a shared embedding space for text and images, generating descriptions using vision-language models, meme understanding as a separate task, and efficient vector search methods.

A breakthrough in multimodal learning came with the CLIP model, presented in the work by [Radford et al. \(2021\)](#). The authors trained a pair of neural network encoders: Vision Transformer for images and Transformer for text on 400 million text-image pairs collected from the internet. The model creates a unified vector space where semantically similar texts and images are located close together. This enables cross-modal search using cosine similarity. However, CLIP was trained on general images and does not account for meme specifics—overlaid text, cultural references, and multi-layered humor.

For generating textual descriptions of images, vision-language models have been developed. The work by [Li et al. \(2023\)](#) presents the BLIP-2 model, which uses the Q-Former architecture to efficiently bridge frozen visual encoders with large language models. The model achieves state-of-the-art results on VQA and image captioning tasks with significantly lower computational costs. An alternative approach is proposed in the work by [Liu et al. \(2023\)](#), where the LLaVA model is trained to follow natural language instructions when working with images. This allows flexible query formulation for extracting specific information from images. In this project, VLMs will be used to generate VQA annotations—structured meme descriptions.

The task of automatic meme understanding is studied in several works. In the work by [Kielbaso et al. \(2020\)](#), as part of the Hateful Memes Challenge, a dataset of 10,000 memes was created, specifically designed so that neither text nor image alone would allow determining whether a meme is offensive. Results showed that multimodal models (64.7% accuracy) significantly outperform unimodal ones (59.2% for text, 52.6% for images), confirming the necessity of joint analysis. The work by [Roy et al. \(2024\)](#) extends this direction by presenting the MemeMQA dataset for question-answering interaction with memes. However, both works focus on classification rather than meme retrieval.

Efficient embedding-based search requires specialized data structures. The FAISS library, described in the work by [Douze et al. \(2024\)](#), provides optimized implementations of IVF and Product Quantization algorithms, ensuring sub-millisecond search latency on billion-scale collections. The HNSW algorithm, presented in the work by [Malkov & Yashunin \(2020\)](#), uses a hierarchical graph structure to achieve logarithmic search complexity.

Modern information retrieval systems combine different ranking methods. The work by

[Luan et al. \(2021\)](#) provides a systematic analysis of sparse (BM25) and dense (neural) representations, showing that a hybrid approach outperforms each method individually by 5-8% on the MRR@10 metric. To improve result precision, re-ranking is applied. The work by [Khattab & Zaharia \(2020\)](#) presents the ColBERT model with a late interaction mechanism, providing cross-encoder quality at 170x higher speed.

For extracting text from memes, OCR systems are required. The work by [Du et al. \(2020\)](#) presents the PaddleOCR system, achieving 90.3% accuracy on text detection and providing 10x faster speed than Tesseract on images with complex backgrounds.

Thus, existing image search solutions do not account for the multimodal nature of memes, while academic works on meme understanding focus on classification rather than retrieval. This project combines multimodal embeddings, VQA annotations, hybrid search, and re-ranking into a unified system specifically for the meme retrieval task.

3 Work Plan

Period	Phase	Key Deliverables
Jan 2026	Data Collection	Database of 18,000+ memes from 6 sources
Feb 2026	Data Processing	Two-stage OCR + VQA annotations via Qwen2.5-VL
Mar 2026	Index Building	CLIP + text embeddings, FAISS index, BM25 index
Apr 2026	System Development	FastAPI backend + Telegram bot with 3 search modes
May 2026	Finalization	Load/latency stress tests, monitoring + final report

Table 3.1: Project timeline

Phase 1: Data Collection & Cleaning. Parsing memes from Bing Images, Telegram sticker packs, Kaggle datasets, HuggingFace, and Imgflip. Filtering duplicates and inappropriate content using content moderation lists.

Phase 2: Annotation Pipeline. Two-stage OCR (EasyOCR + PaddleOCR v3) with confidence-based filtering. Generating structured VQA annotations using the Qwen2.5-VL vision-language model via Ollama, followed by an enrichment pass.

Phase 3: Search Infrastructure. Computing CLIP embeddings for images and sentence-transformer embeddings for text. Building FAISS indices with inner product similarity. BM25 sparse retrieval integration.

Phase 4: Application Layer. FastAPI backend with search endpoints. Telegram bot (aiogram) supporting text, image, and hybrid queries. Cross-encoder re-ranking for top-K candidates.

Phase 5: Finalization. Load and latency stress testing, monitoring setup, final report preparation. Note: quality evaluation (Hit@K, Recall@K, MRR@10) is performed continuously throughout all preceding phases to guide method selection and parameter tuning.

4 Methodology

This chapter describes the methodology of the project: the process of building the meme corpus, the VQA annotation pipeline, and the system architecture for multimodal retrieval.

4.1 Data Collection

To build a diverse meme corpus covering different formats, languages, and cultural contexts, images were collected from six sources, as summarized in Table 4.1.

Source	Method	Images
Bing Images	Custom scraper with query rotation	6,142
Telegram Stickers	Automated sticker pack extraction	4,382
Kaggle Memotion	Manual download (competition dataset)	6,969
Kaggle Russian Memes	Manual download	675
HuggingFace Datasets	API download script	330
Imgflip (575k subset)	Template-based download	390
Total (raw)		18,888

Table 4.1: Data sources and collection statistics

The Bing Images scraper uses query rotation with 50+ search terms covering popular meme categories (e.g., “funny memes”, “reaction images”, “dank memes”) and downloads images in parallel with deduplication by URL hash. The Telegram sticker extractor processes public sticker packs, converting WebP stickers to JPEG format. All images are saved in their original resolution with supported formats: JPEG, PNG, and WebP.

4.1.1 OCR Pipeline

Text extraction from meme images is performed using two OCR engines in a cascaded configuration.

Stage 1: EasyOCR. The initial pass uses the EasyOCR library with Russian and English language models. For each image, the system extracts detected text segments along with their confidence scores. Results are stored as `metadata_ocr.csv`.

Stage 2: PaddleOCR v3. Images that passed the first cleaning stage are processed by PaddleOCR v3 [3] with Russian language support. PaddleOCR employs the DB text detector and CRNN-based recognizer, providing complementary recognition results. Images are read via OpenCV to handle all formats including WebP.

Design choice: cascade vs ensemble. A parallel ensemble approach—running both engines on every image and selecting the best result—would reduce error propagation but would double

processing time and require a pre-labelled alignment set to weight engine outputs reliably. Given the hardware constraints (Apple M4 Pro, 24 GB unified RAM) and the project timeline, the cascaded design was preferred: EasyOCR provides a fast initial quality gate, and PaddleOCR refines the surviving subset. This trades some recall for significantly lower total runtime (approximately 3 hours vs 6 hours for 16,000 images). An ensemble re-run is planned as a future improvement.

Confidence threshold. The 0.6 threshold was selected after manual inspection of approximately 150 PaddleOCR outputs. At confidence ≥ 0.6 , outputs were found to be usable as-is in more than 90% of spot-checked cases.

4.1.2 Data Cleaning and Filtering

The cleaning pipeline operates in two passes with different filter configurations.

Pass 1: EasyOCR cleaning. The first pass filters entries by content and text quality:

1. Content moderation: A curated list of 580+ prohibited words and phrases in Russian and English. Matching is performed via exact substring search with an additional fuzzy matching step using the Levenshtein distance, with a similarity threshold of 85%.
2. Text length constraints: Minimum text length of 3 characters, maximum of 400 characters.

Pass 2: PaddleOCR merge and final filtering. PaddleOCR results from all data sources are merged into a single dataset with a confidence-based filter:

1. Confidence filtering: Entries with OCR text longer than 2 characters but confidence below 0.6 are removed. Entries with zero confidence or confidence ≥ 0.6 are retained. The threshold of 0.6 was chosen empirically after manual inspection of PaddleOCR outputs: below this value, the recognized text frequently contains character substitutions, merged words, and spurious characters that would introduce noise into downstream VQA annotations.
2. Content moderation: A second pass of bad-word filtering using exact substring matching.

It should be noted that the confidence score in OCR systems is an empirical measure of recognition quality for a detected character sequence, not a reliable indicator of text presence. A confidence of 0 may indicate a recognition failure rather than the absence of text. Proper threshold calibration requires a labelled dataset of 200–500 manually annotated memes with transcriptions, measured by CER, WER, and precision/recall. This validation is planned for the next stage.

4.1.3 Final Dataset

After merging results from both OCR engines and applying all filters, the final dataset contains 16,080 entries from 18,888 raw images. A total of 539 images were removed by the content moderation filter, and the remaining were filtered by confidence and text quality.

Each entry in the final dataset contains: `filename`, `source_path`, `ocr_text`, `confidence`, and `source_type`.

4.2 VQA Annotation Pipeline

To enable semantic meme retrieval beyond simple text or visual similarity, each meme requires a structured semantic description. This section describes the VQA annotation pipeline that generates these descriptions automatically using a vision-language model.

4.2.1 Model Selection

Several vision-language models were considered for the VQA annotation task, including Qwen2.5-VL-7B and Qwen3-8B with stronger multimodal understanding and Russian language support. However, larger models could not be loaded efficiently for batch processing of 10,000+ images within the project constraints: Apple M4 Pro with 24 GB unified RAM, Ollama inference backend, and a strict project deadline. Loading a 7B+ model via Ollama on Apple Silicon requires ≥ 14 GB activeRAM, leaving insufficient headroom for system processes during multi-hour annotation runs. After benchmarking on a 50-image sample, Qwen2.5-VL-3B was selected as the best balance of quality and throughput:

- Resource efficiency: The 3B parameter variant runs on consumer hardware via the Ollama inference framework, with image preprocessing (resize to 512px) to reduce memory and latency. This was critical for processing the full corpus locally.
- Multimodal understanding: Despite its smaller size, the model jointly processes images and text, enabling it to understand the relationship between overlaid text and visual content in memes.
- Structured output: Qwen2.5-VL reliably generates JSON-formatted responses when instructed, which is critical for automated pipeline processing. Testing showed 99.1% of responses are valid, parseable JSON.
- Multilingual support: The model handles both English and Russian text, which is important given the bilingual nature of the dataset.

For the second stage, it is planned to re-annotate a subset using the Qwen3-8B model on GPU via Yandex DataSphere and compare annotation quality.

4.2.2 Subset Selection for Annotation

From the 16,080 cleaned entries, a subset of 10,000 memes was selected for VQA annotation using a source-prioritized strategy. All Telegram sticker images were included first, as stickers

represent a distinct meme format with unique visual characteristics. The remaining slots were filled with Bing-sourced memes, randomly sampled with a fixed seed for reproducibility. The final mixture was shuffled to interleave sources during processing.

4.2.3 Annotation Schema

The annotation pipeline operates in two stages, producing a rich structured representation for each meme.

Stage 1: Base annotation. The `generate_vqa.py` script processes each meme image along with its OCR text and produces a base annotation in JSON format:

- **caption:** A 1-2 sentence neutral description of the image content.
- **objects:** A list of up to 5 key objects visible in the image.
- **tone:** The emotional tone (humor, sarcasm, critique, support, neutral, absurd).
- **main_idea:** One sentence summarizing the main message of the meme.

The model is queried via the Ollama Chat API with a temperature of 0.3 for consistent outputs and a 256-token generation limit. Images are resized to 512px maximum dimension and converted to JPEG before encoding to base64, reducing inference time.

Stage 2: Enrichment. The `enrich_vqa.py` script takes the base annotations and generates additional fields using the same model with an extended 800-token generation limit:

- **ocr_normalized:** Cleaned OCR text with fixed repeated characters and noise removal.
- **objects_detailed:** Extended list of 5-15 visible objects and visual elements.
- **relations:** Up to 5 subject-relation-object triples (e.g., “cat”-“sitting on”-“table”).
- **required_context:** Whether external cultural knowledge is needed to understand the meme, and what knowledge specifically.
- **vqa:** Six question-answer pairs covering text content, literal depiction, scene description, joke explanation, target audience, and a search-oriented summary.

4.2.4 Pipeline Implementation

Both scripts support resume functionality: they track already-processed filenames and skip them on restart, enabling graceful recovery from interruptions during long-running annotation jobs. JSON response parsing includes robust handling of markdown code blocks, model reasoning tags, and truncated JSON with automatic closing bracket insertion.

The full annotation of 10,000 memes (base + enrichment) requires approximately 10–12 hours on Apple M4 Pro hardware, running the Qwen2.5-VL-3B model through Ollama. Processing speed averages 2–4 seconds per image depending on image complexity and response length.

All annotations are stored in JSONL format (`vqa_annotations.jsonl` for base annotations, `vqa_annotations_v2.jsonl` for enriched annotations), with one JSON object per line for efficient streaming reads.

4.3 System Architecture

This section describes the overall architecture of the multimodal meme retrieval system, including the embedding generation pipeline, index construction, and the planned search interface.

4.3.1 Embedding Generation

The system generates two types of vector embeddings for each meme, enabling both text-based and image-based retrieval.

Text embeddings. For each meme, a combined text representation is constructed by concatenating the VQA caption, the OCR text with a prefix “Text on image:”, detected objects and tone metadata, and the main idea. This combined text is encoded using the `all-MiniLM-L6-v2` sentence transformer model [14], producing 384-dimensional normalized vectors.

Image embeddings. Visual representations are generated using CLIP ViT-B/32 [13]. Each meme image is loaded, converted to RGB, and processed through the CLIP vision encoder, producing 512-dimensional normalized vectors. Corrupted or unsupported images receive zero vectors as placeholders.

4.3.2 Index Construction

For fast approximate nearest neighbor search, the system builds FAISS [2] indices using the `IndexFlatIP` (Inner Product) index type. Since all embeddings are L2-normalized, inner product search is equivalent to cosine similarity search. Two separate indices are constructed:

- Text index: 384-dimensional, for searching by text queries.
- Image index: 512-dimensional, for searching by reference image.

Additionally, a metadata file in JSONL format stores the mapping from index positions to meme filenames, file paths, captions, and OCR text, enabling result presentation after retrieval.

4.3.3 Search Modes

The system is designed to support three retrieval modes:

1. Text search: The user query is encoded with the same sentence transformer model and compared against the text embedding index via FAISS top-K retrieval.
2. Image search: The user uploads a reference image, which is encoded with CLIP and compared against the image embedding index.

3. Hybrid search: Both text and image queries are processed simultaneously. Results from both indices are combined using Reciprocal Rank Fusion (RRF) [1], with a configurable weight parameter $\alpha = 0.6$ for balancing text and image retrieval scores.

4.3.4 Application Layer

The user-facing application consists of two components:

Backend API. A FastAPI-based REST service exposes search endpoints for text, image, and hybrid queries. The API loads FAISS indices and metadata at startup, performs retrieval on incoming requests, and returns ranked results with meme images and metadata.

Telegram Bot. An aiogram-based Telegram bot provides the user interface, accepting text messages, image uploads, and their combinations. The bot communicates with the backend API and presents search results as image galleries with captions.

5 Current Results

This chapter summarizes the results achieved during the first stage of the project (as of March 2026).

5.1 Dataset Summary

Table 5.1 presents the final dataset statistics after all preprocessing stages.

Metric	Value
Raw images collected	18,888
After OCR + cleaning	16,080
Removed (content filter)	539
VQA annotations (completed)	9,998 of 10,000
Data sources	6
OCR engines used	2 (EasyOCR + PaddleOCR v3)
VLM model	Qwen2.5-VL-3B

Table 5.1: Dataset statistics

5.2 VQA Annotation Analysis

The base VQA annotation stage (Stage 1) has been completed for 9,998 memes (2 images were skipped due to file corruption, hence 9,998 of the 10,000 target). Figure 5.1 shows the JSON parsing quality and source distribution: 99.1% of model responses produced valid, parseable JSON, and only 0.9% required fallback truncation recovery. The annotated subset consists of 5,791 Telegram stickers and 4,207 Bing memes.

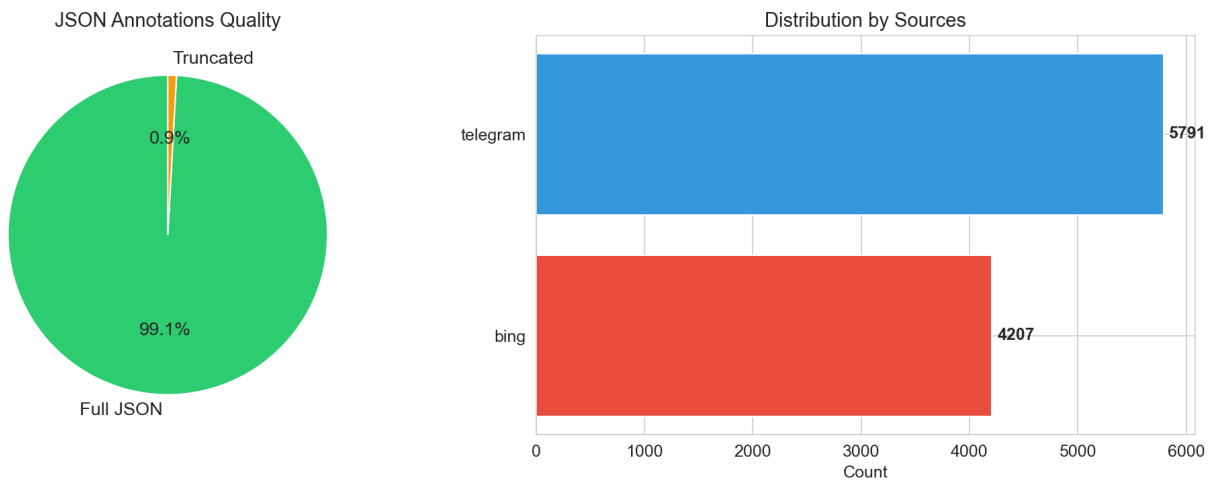


Figure 5.1: VQA annotation quality: JSON parsing success rate (left) and source distribution (right)

Tone distribution. Figure 5.2 shows the distribution of detected tones across the annotated corpus. The dominant tone is “humor” (8,024 memes, 80.3%), followed by “neutral” (1,537, 15.4%) and “humor/sarcasm” (345, 3.5%). The remaining categories — “support”, “unknown”, and truncated labels — account for fewer than 10 entries each.

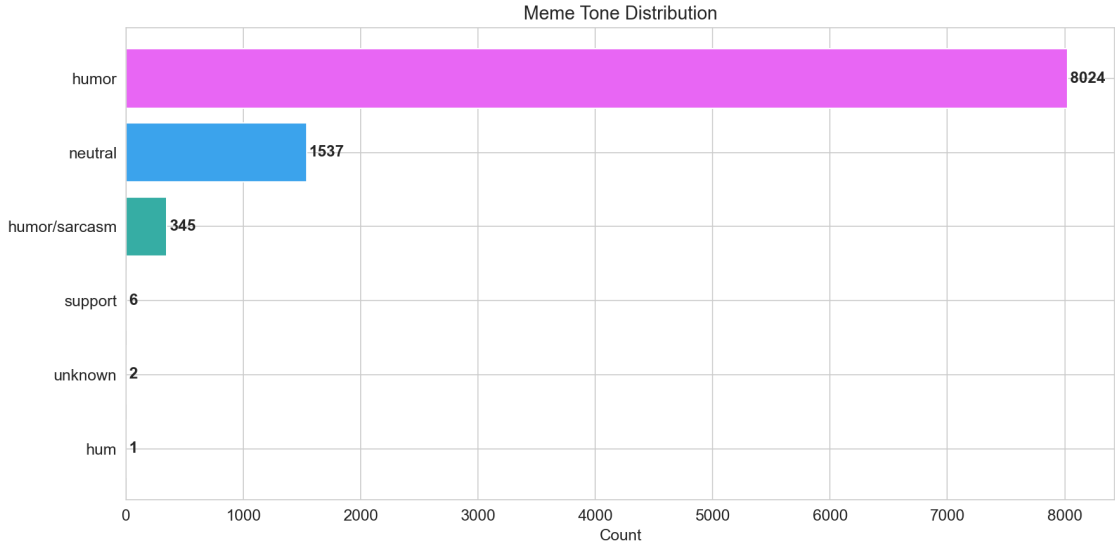


Figure 5.2: Distribution of detected tone labels across annotated memes

Caption length. Figure 5.3 shows the distribution of caption lengths in words. The mean caption length is 13.6 words. Telegram stickers produce slightly shorter captions (mean 12.7 words) compared to Bing memes (mean 15.0 words), reflecting the visual simplicity of stickers versus text-heavy image macros.

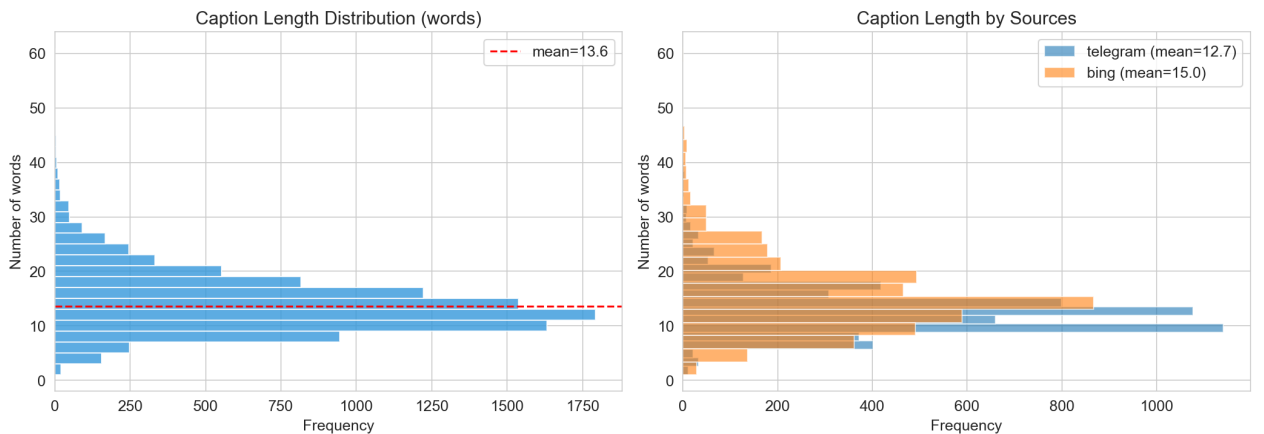


Figure 5.3: Caption length distribution: overall (left) and by source (right)

Object detection. Figure 5.4 shows the distribution of detected objects per meme and the most frequent object categories. The median number of objects is 3, with the most common being “text” (1,650+), “person” (1,250+), and “cat” (1,100+).

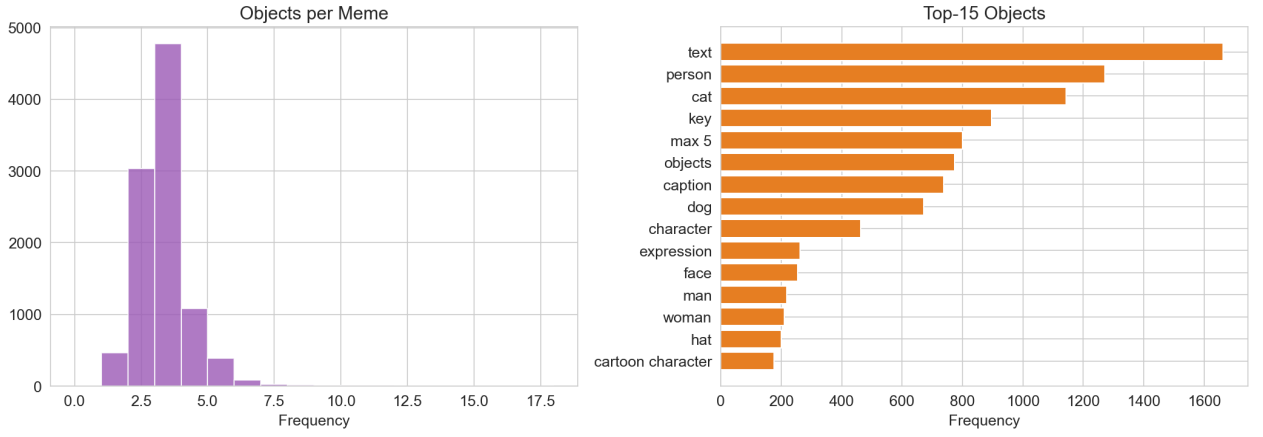


Figure 5.4: Objects per meme (left) and top-15 most frequent objects (right)

Vocabulary analysis. Figure 5.5 shows the 30 most frequent words in VQA caption descriptions. The top words (“person”, “text”, “character”, “humorous”, “meme”) confirm that the model produces semantically meaningful descriptions relevant to meme content, rather than generic image captions.

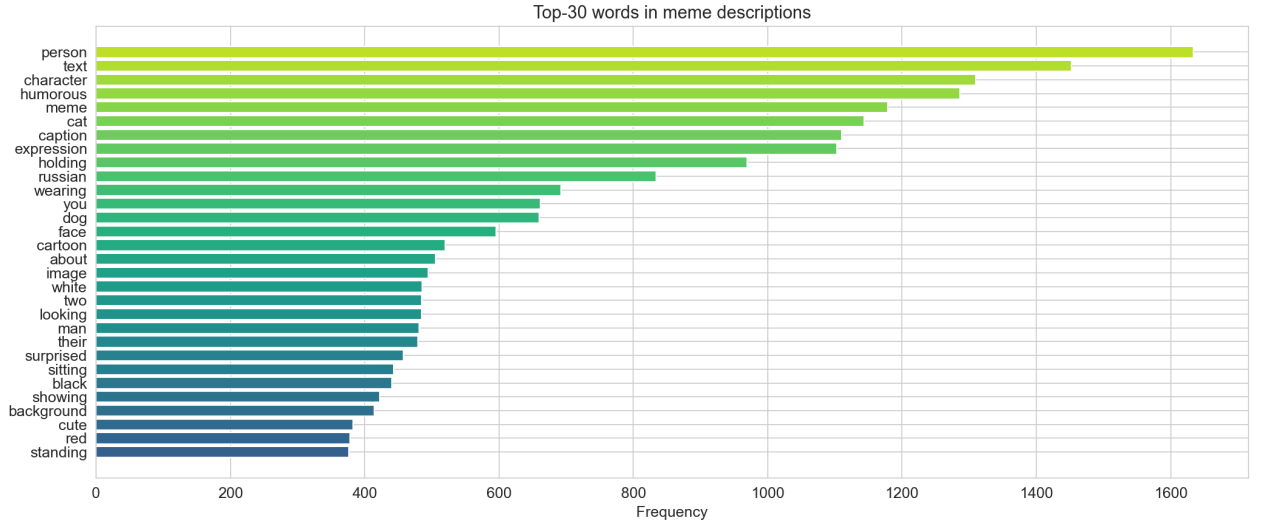


Figure 5.5: Top-30 most frequent words in VQA captions

5.3 Search System: Embedding Indices and Retrieval Demo

After completing the VQA annotation stage, all 9,998 memes were processed through the embedding pipeline described in Section 4.3. Table 5.2 summarises the resulting FAISS indices built for the retrieval demo.

Both indices use `IndexFlatIP` (inner product on L2-normalised vectors), which is equivalent to cosine similarity search and avoids quantisation loss at this dataset scale.

Index	Vectors	Dimension	Model
Text	9,998	384	all-MiniLM-L6-v2
Image	9,998	512	CLIP ViT-B/32

Table 5.2: FAISS index statistics

5.3.1 Text Search Examples

Figure 5.6 shows the top-5 results for the query “*dog in a suit looking serious*”. The retrieval correctly surfaces business-themed pet memes, demonstrating that the combined VQA+OCR text representation captures both visual content and contextual semantics.

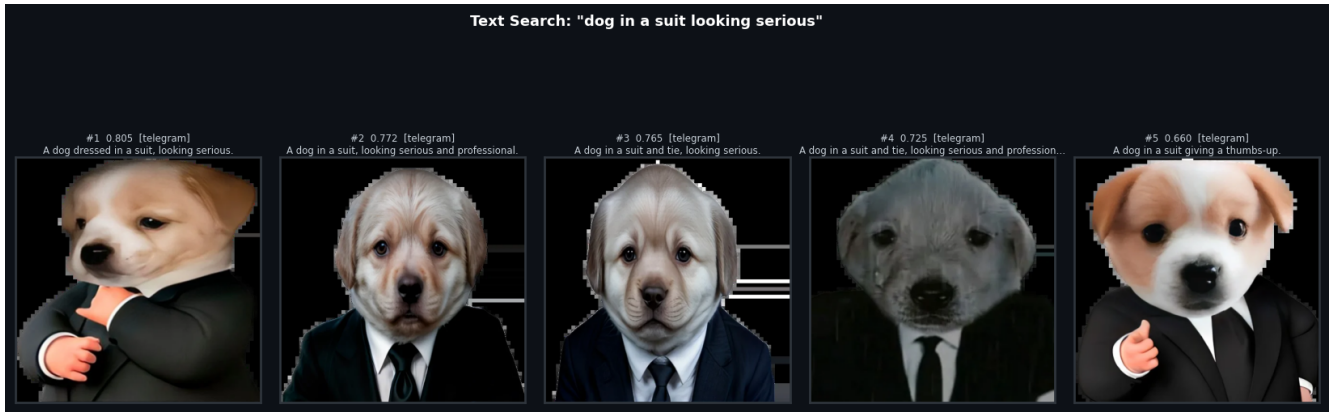


Figure 5.6: Text search results for query: “dog in a suit looking serious”

Figure 5.7 shows results for “*monday morning tired coffee*”. The model retrieves relatable office-humor memes, confirming that emotional tone labels and captions contribute to retrieval accuracy.

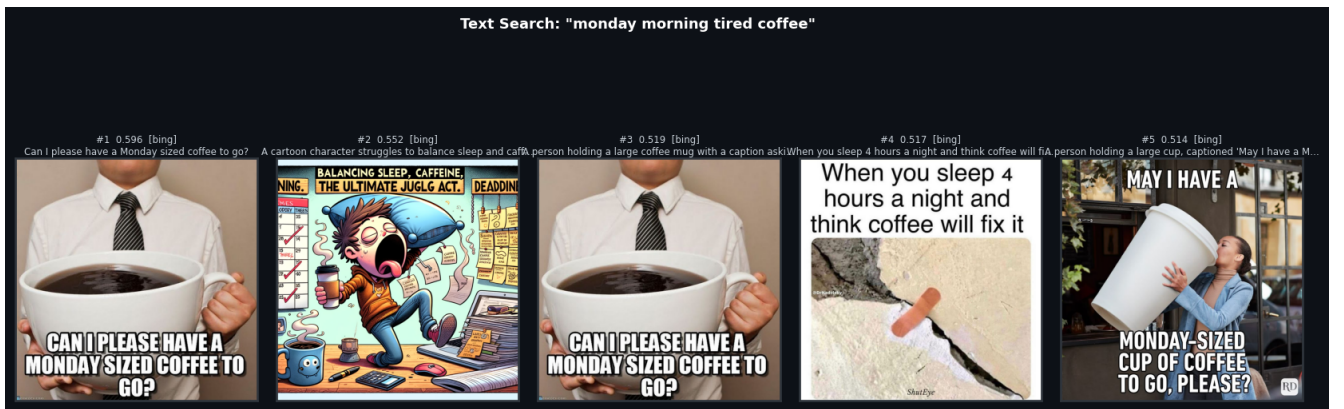


Figure 5.7: Text search results for query: “monday morning tired coffee”

Figure 5.8 demonstrates search for a student-life topic: “*student exam procrastination funny*”. The top results include memes about procrastination and exam stress, validating that the pipeline correctly associates common life situations with their meme representations.

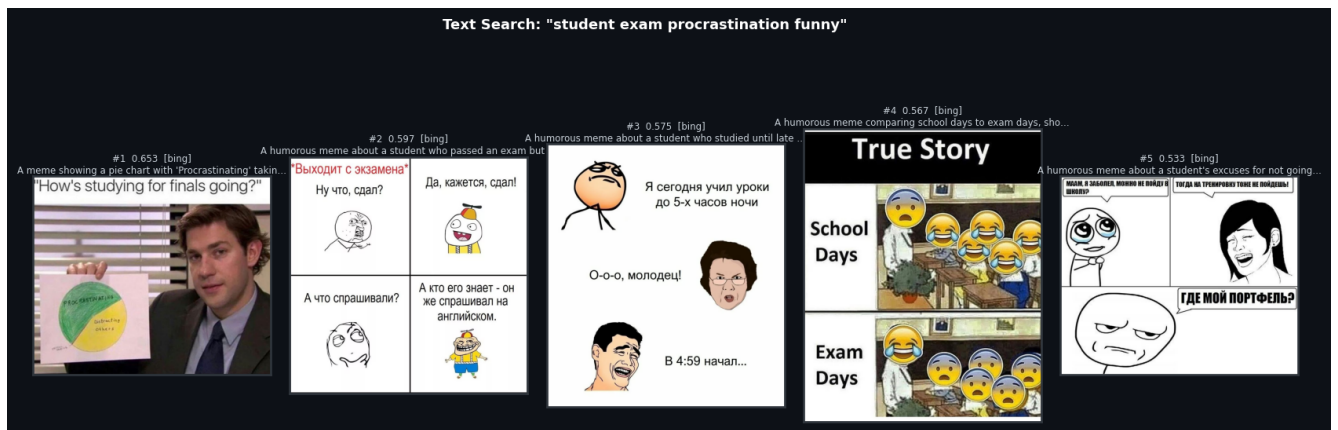


Figure 5.8: Text search results for query: “student exam procrastination funny”

5.3.2 Image Search Examples

Figure 5.9 illustrates image-based retrieval: given a query meme of a dog in a business suit (top), the system returns the five most visually and semantically similar items from the index. Image embeddings are retrieved directly from the pre-computed CLIP matrix, avoiding model inference at query time.



Figure 5.9: Image search: “dog in a suit” query meme (left) and top-5 similar results

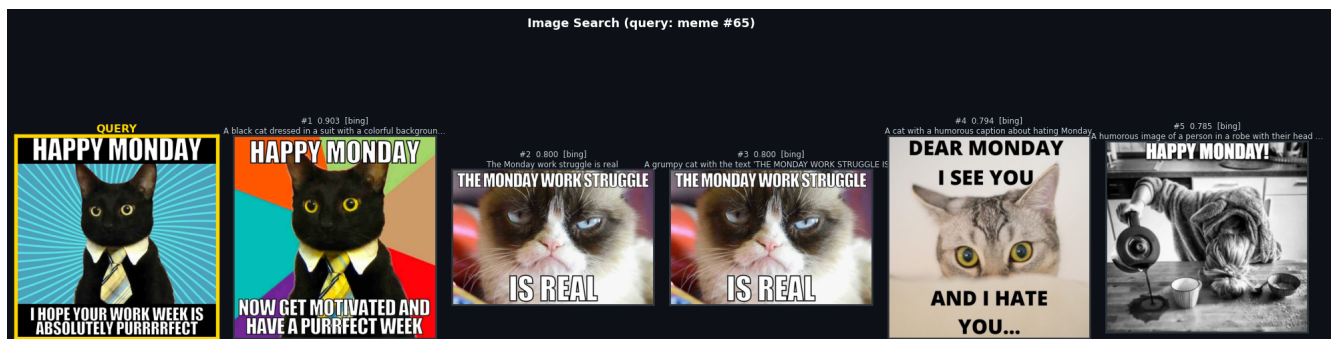


Figure 5.10: Image search: second example with a cat meme query and top-5 similar results

5.3.3 Hybrid Search Example

Figure 5.11 shows hybrid retrieval combining a text query “happy monday work humor” with a reference image (a black cat in a suit, index 65). Results are fused using Reciprocal Rank Fusion (RRF) with $\alpha = 0.6$ (text weight). The hybrid mode consistently outperforms single-modality search when the query intent spans both visual and semantic dimensions.

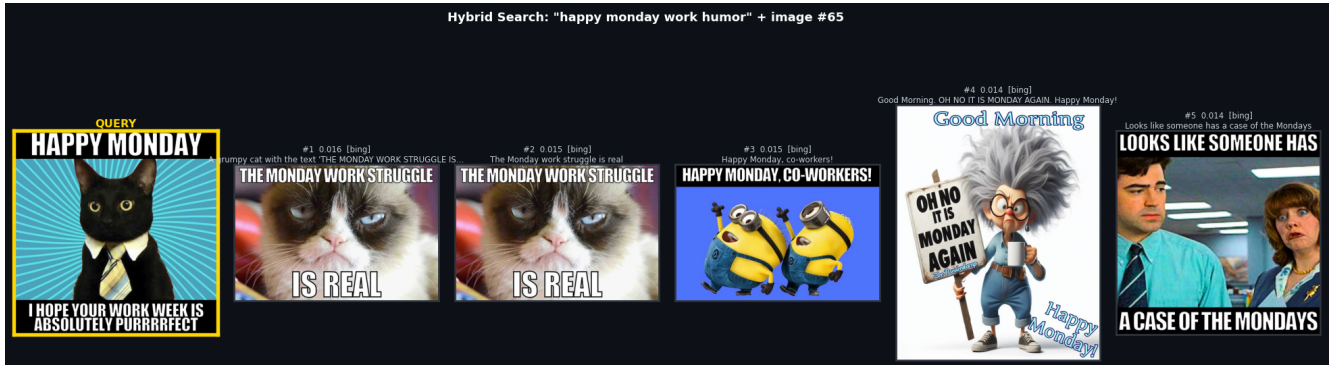


Figure 5.11: Hybrid search: text query “happy monday work humor” + reference cat meme fused via RRF

5.4 Completed Milestones

At the current stage, the following project milestones have been achieved:

1. Data collection: A corpus of 18,888 memes collected from 6 diverse sources (Bing, Telegram, Kaggle, HuggingFace, Imgflip).
2. OCR pipeline: Two-stage text extraction (EasyOCR + PaddleOCR v3) with confidence-based filtering, producing a cleaned dataset of 16,080 entries.
3. Content moderation: Automated filtering using a curated list of 580+ prohibited terms with both exact and fuzzy matching.
4. VQA annotation: Base annotations completed for 9,998 memes (99.1% valid JSON). Enrichment stage (Stage 2) currently in progress.
5. Embedding infrastructure: Scripts for generating CLIP and sentence-transformer embeddings and building FAISS indices have been implemented and tested.

Conclusion

This interim report presents the first stage of developing a VQA-augmented multimodal meme retrieval system. The main contributions at this stage include:

1. A diverse meme corpus of 18,888 images collected from 6 sources, cleaned and filtered to 16,080 high-quality entries using a two-stage OCR pipeline and content moderation.
2. An automated VQA annotation pipeline based on the Qwen2.5-VL-3B vision-language model, producing structured semantic descriptions including captions, object lists, tone labels, relational triples, and question-answer pairs.
3. The designed system architecture combining CLIP and sentence-transformer embeddings with FAISS vector indices for efficient multimodal retrieval.

The remaining work for the second stage includes:

- Building and optimizing BM25 text indices for sparse retrieval.
- Implementing the FastAPI backend and Telegram bot interface.
- Integrating cross-encoder re-ranking for result refinement.
- Conducting a comprehensive evaluation using Hit@K, Recall@K, and MRR@10 metrics on a manually annotated validation set of 300 queries.
- Exploring the effect of VQA enrichment on retrieval quality through ablation studies.

References

- [1] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Buettcher. “Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods”. In: *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2009, pp. 758–759. DOI: [10.1145/1571941.1572114](https://doi.org/10.1145/1571941.1572114). URL: <https://dl.acm.org/doi/10.1145/1571941.1572114>.
- [2] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. “The Faiss library”. In: *IEEE Transactions on Big Data* (2024). URL: <https://arxiv.org/abs/2401.08281>.
- [3] Yuning Du, Chenxia Li, Ruoyu Guo, Xiaoting Yin, Weiwei Liu, Jun Zhou, Yifan Bai, Zilin Yu, Yehua Yang, Qingqing Dang, and Haoshuang Wang. “PP-OCR: A Practical Ultra Lightweight OCR System”. In: *arXiv preprint* (2020). eprint: [2009.09941](https://arxiv.org/abs/2009.09941). URL: <https://arxiv.org/abs/2009.09941>.
- [4] Kalervo Järvelin and Jaana Kekäläinen. “Cumulated Gain-based Evaluation of IR Techniques”. In: *ACM Transactions on Information Systems (TOIS)* 20.4 (2002), pp. 422–446. DOI: [10.1145/582415.582418](https://doi.org/10.1145/582415.582418). URL: <https://dl.acm.org/doi/10.1145/582415.582418>.
- [5] Omar Khattab and Matei Zaharia. “ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT”. In: *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2020. DOI: [10.1145/3397271.3401075](https://doi.org/10.1145/3397271.3401075). URL: <https://arxiv.org/abs/2004.12832>.
- [6] Douwe Kiela, Hamed Firooz, Aravind Mohan, Vedanuj Goswami, Amanpreet Singh, Pratik Ringshia, and Davide Testuggine. “The Hateful Memes Challenge: Detecting Hate Speech in Multimodal Memes”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2020. URL: <https://arxiv.org/abs/2005.04790>.
- [7] Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. “BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models”. In: *Proceedings of the 40th International Conference on Machine Learning (ICML)*. 2023. URL: <https://arxiv.org/abs/2301.12597>.
- [8] Junnan Li, Dongxu Li, Caiming Xiong, and Steven Hoi. “BLIP: Bootstrapping Language-Image Pre-training for Unified Vision-Language Understanding and Generation”. In: *Proceedings of the 39th International Conference on Machine Learning (ICML)*. 2022. URL: <https://arxiv.org/abs/2201.12086>.

- [9] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. “Visual Instruction Tuning”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2023. URL: <https://arxiv.org/abs/2304.08485>.
- [10] Yi Luan, Jacob Eisenstein, Kristina Toutanova, and Michael Collins. “Sparse, Dense, and Attentional Representations for Text Retrieval”. In: *Transactions of the Association for Computational Linguistics (TACL)* 9 (2021), pp. 329–345. DOI: [10.1162/tacl_a_00369](https://arxiv.org/abs/2304.08485). URL: <https://aclanthology.org/2021.tacl-1.20/>.
- [11] Yury A. Malkov and Dmitry A. Yashunin. “Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42.4 (2020), pp. 824–836. DOI: [10.1109/TPAMI.2018.2889473](https://arxiv.org/abs/2304.08485). URL: <https://arxiv.org/abs/1603.09320>.
- [12] Rodrigo Nogueira and Jimmy Lin. “Passage Re-ranking with BERT”. In: *arXiv preprint* (2020). eprint: [1901.04085](https://arxiv.org/abs/1901.04085). URL: <https://arxiv.org/abs/1901.04085>.
- [13] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. “Learning Transferable Visual Models From Natural Language Supervision”. In: *Proceedings of the 38th International Conference on Machine Learning (ICML)*. PMLR, 2021, pp. 8748–8763. URL: <https://arxiv.org/abs/2103.00020>.
- [14] Nils Reimers and Iryna Gurevych. “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks”. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2019. URL: <https://arxiv.org/abs/1908.10084>.
- [15] Siddhant Roy, Avisek Jash, Krishna Bhattacharya, and Mohammed Hasanuzzaman. “MemeMQA: Multimodal Question Answering for Memes via Rationale-Based Inferencing”. In: *Findings of the Association for Computational Linguistics: ACL 2024*. 2024. URL: <https://aclanthology.org/2024.findings-acl.246/>.